SDA Week 2
Lecture # 5 & 6

Syed Ahmed Khan
syed.ahmed@nu.edu.pk

What is a Domain Model?

- Illustrates meaningful conceptual classes in problem domain
- Represents real-world concepts, not software components
- Software-oriented class diagrams will be developed later, during design

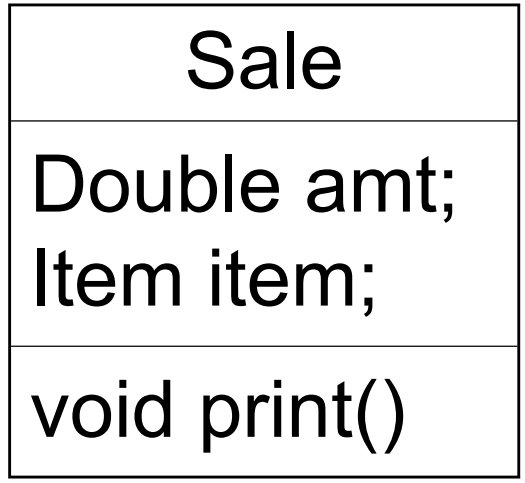
A Domain Model is Conceptual, not a Software Artifact

Conceptual Class:

vs.

What's the difference?

Software Artifacts:



Why do a domain model?

- Gives a conceptual framework of the things in the problem space
- Helps you think – focus on semantics
- Provides a glossary of terms – noun based
- It is a static view - meaning it allows us convey time invariant business rules
- Foundation for use case/workflow modelling
- Based on the defined structure, we can describe the state of the problem domain at any time.

Features of a domain model

- **Conceptual classes** – each conceptual class denotes a type of object.
- **Attributes** – an attribute is the description of a named slot of a specified type in a domain class; each instance of the class separately holds a value.
- **Associations** – an association is a relationship between two (or more) domain classes that describes links between their object instances. Associations can have roles, describing the multiplicity and participation of a class in the relationship.

How to create Domain Models?

- Find the conceptual classes.
- Draw them as classes in a UML class diagram.
- Add associations and attributes

Conceptual classes?

- The domain model illustrates conceptual classes or vocabulary in the domain. Informally, a conceptual class is an idea, thing, or object.
- Each conceptual class denotes a type of object. It is a descriptor for a set of things that share common features. Classes can be:-
- ***Business objects*** **-** represent things that are manipulated in the business e.g. *Order*.
- ***Real world objects*** – things that the business keeps track of e.g. *Contact*, *Site*.
- ***Events that transpire*** **-** e.g. *sale* and *payment*.

How to Identify Conceptual Classes

- Reuse an existing domain model
  - There are many published, well-crafted domain models.
- Use a conceptual class category list
  - Make a list of all candidate conceptual classes
- Identify noun phrases
  - Identify nouns and phrases in textual descriptions of a domain (use cases, or other documents)

Conceptual Class Category List

- Physical or tangible objects
  - Register, Airplane
- Specifications, or descriptions of things
  - ProductSpecification, FlightDescription
- Places
  - Store, Airport
- Transactions
  - Sale, Payment, Reservation
- Transaction items
  - SalesLineItem
- Roles of people
  - Cashier, Pilot
- Containers of other things
  - Store, Hangar, Airplane
- Things in a container
  - Item, Passenger
- Computer or electro mechanical systems
  - CreditPaymentAuthorizationSystem, AirTrafficControl
- Catalogs
  - ProductCatalog, PartsCatalog
- Organizations
  - SalesDepartment, Airline

Where identify conceptual classes from noun phrases (NP)

- Vision and Scope, Glossary and Use Cases are good for this type of linguistic analysis
- However:
- Words may be ambiguous or synonymous
- Noun phrases may also be attributes or parameters rather than classes:
  - If it stores state information or it has multiple behaviors, then it's a class
  - If it's just a number or a string, then it's probably an attribute

Where identify conceptual classes from noun phrases (NP)

The ATM verifies whether the customer's card number and PIN are correct.
S V O O O
If it is, then the customer can check the account balance, deposit cash, and withdraw cash.
S V O V O V O
Checking the balance simply displays the account balance.
S O V O
Depositing asks the customer to enter the amount, then updates the account balance.
S V O V O V O
Withdraw cash asks the customer for the amount to withdraw; if the account has enough cash,
S O V O O V S V O
the account balance is updated. The ATM prints the customer's account balance on a receipt.
O V S V O O

Where identify conceptual classes from noun phrases (NP)

Use Case Scenario: Customer confirms items in shopping cart. Customer provides payment and address to process sale. System validates payment and responds by confirming order, and provides order number that Customer can use to check on order status. System will send Customer a copy of order details by email.

Finding Conceptual Classes with Noun Phrase Identification

- **Customer** arrives at a **POS checkout** with **goods** and/or **services** to purchase.
- **Cashier** starts a new **sale**.
- **Cashier** enters **item identifier**.
- System records **sale line item** and presents **item description**, **price**, and running **total**. Price calculated from a set of price rules. Cashier repeats steps 2-3 until indicates done.

Finding Conceptual Classes with Noun Phrase Identification

- System presents total with **taxes** calculated.
- Cashier tells Customer the total, and asks for **payment**.
- Customer pays and System handles payment.
- System logs the completed **sale** and sends sale and payment information to the external **Accounting** (for accounting and **commissions**) and **Inventory** systems (to update inventory).
- System presents **receipt**.
- Customer leaves with receipt and goods (if any).

Example Conceptual Classes found

Class names

- **Class Name** creates the vocabulary of our analysis
  - Use nouns as class names, think of them as simple agents
  - Verbs can also be made into nouns, if they are maintain state
  - E.g., “reads card” suggests **CardReader**, managing bank cards
- Use pronounceable names:
  - If you cannot read aloud, it is not a good name
- Use capitalization to initialize Class names and demarcate multi-word nar
  - E.g., CardReader rather than CARDREADER or card_reader
  - Why do most OO developers prefer this convention?
- Avoid obscure, ambiguous abbreviations
  - E.g., is TermProcess something that terminates
  - or something that runs on a terminal?
- Try *not* to use digits within a name, such as CardReader2
  - Better for instances than classes of objects

Steps to create a Domain Model

1. Identify candidate conceptual classes
2. Draw them in a UML domain model
3. Add associations necessary to record the relationships that must be retained
4. Add attributes necessary for information to be preserved
5. Use existing names for things, the vocabulary of the domain

2.Adding Attributes

- An attribute is a logical data value of an object.
- Include the following attributes in a domain model: Those for which the requirements suggest a need to remember information.
- An attribute can be a more complex type whose structure is unimportant to the problem, so we treat it like a simple type
- UML Attributes Notation: Attributes are shown in the second compartment of the class box

3.Adding Association

- An association is a relationship between classes that indicates some meaningful and interesting connection.
- In the UML, associations are defined as “the semantic relationship between two or more classifiers that involve connections among their instances.”

Common Associations

- A is subpart/member of B. (SaleLineItem-Sale)
- A uses or manages B. (Cashier –Register, Pilot-airplane)
- A communicates with B. (Student -Teacher)
- A is transaction related to B. (Payment -Sale)
- A is next to B. (SaleLineItem-SaleLineItem)
- A is owned by B. (Plane-Airline)
- A is an event related to B. (Sale-Store)

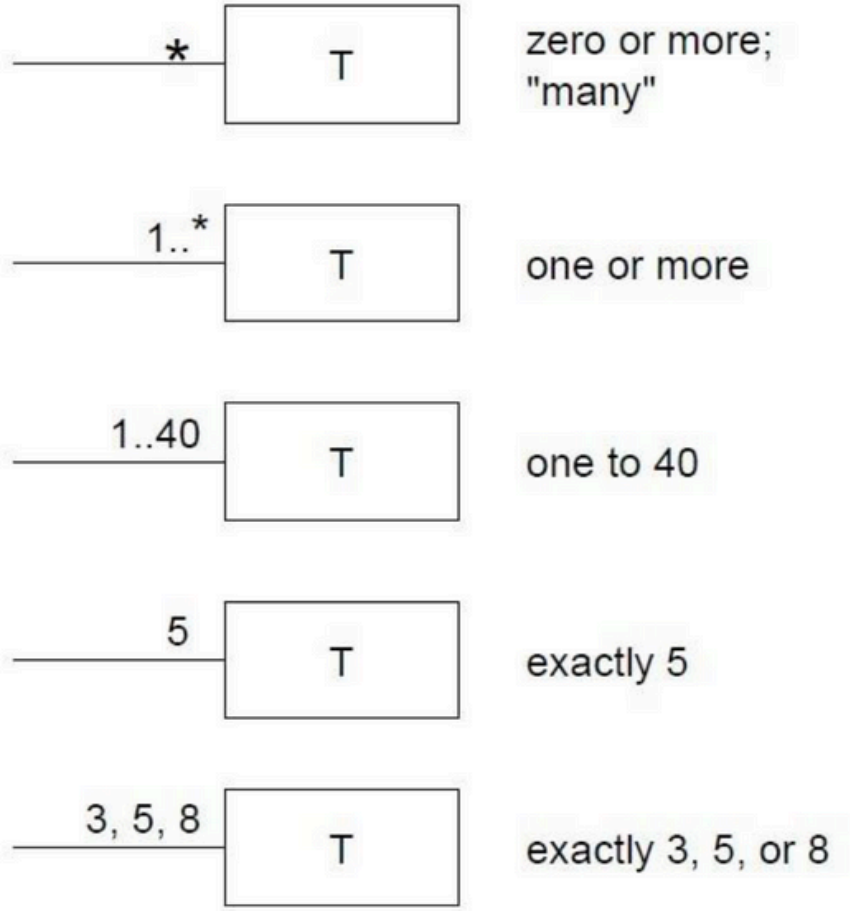
Table 9.2. Common Associations List.

Category	Examples
A is a transaction related to another transaction B	*CashPaymentSale* *CancellationReservation*
A is a line item of a transaction B	*SalesLineItemSale*
A is a product or service for a transaction (or line item) B	*ItemSalesLineItem (or Sale)* *FlightReservation*
A is a role related to a transaction B	*CustomerPayment* *PassengerTicket*
A is a physical or logical part of B	*DrawerRegister* *SquareBoard* *SeatAirplane*
A is physically or logically contained in/on B	*RegisterStore, ItemShelf* *SquareBoard* *PassengerAirplane*
A is a description for B	*ProductDescriptionItem* *FlightDescriptionFlight*
A is known/logged/recorded/reported/captured in B	*SaleRegister* *PieceSquare* *ReservationFlightManifest*
A is a member of B	*CashierStore* *PlayerMonopolyGame* *PilotAirline*
A is an organizational subunit of B	*DepartmentStore* *MaintenanceAirline*

Roles and Multiplicity

- Each end of an association is called a role.
- Multiplicity defines how many instances of a class A can be associated with one instance of a class B.
- e.g.: a single instance of a Store can be associated with “many”(zero or more) Item instances.

Some examples of Multiplicity



Monopoly Game Example

Monopoly is a board game played by two to eight players. A Monopoly game is played using exactly two dice and one board. Each player has a name and owns exactly one playing piece. The board consists of forty squares arranged in a fixed order. During the game, each player's piece is always located on exactly one square on the board. Players take turns rolling the dice and moving their pieces according to the face values shown on the dice. The game continues until it is finished.

Monopoly Game Example

Monopoly Game is a **board** game played by two to eight **Players**. Each Monopoly Game is played using exactly two **Dice** and one Board. Each Player has a **name** and owns exactly one **Piece**. The Board consists of forty **Squares** arranged in a fixed order. During the Game, each Piece is always located on exactly one **Square**. Players take turns rolling the Dice and moving their Pieces according to the **face values** shown on the Dice. The Game continues until it is finished.

Identify Candidate Conceptual Classes

- Monopoly game
- Player
- Board
- Square
- Die
- Dice
- Piece
- Name
- Face value

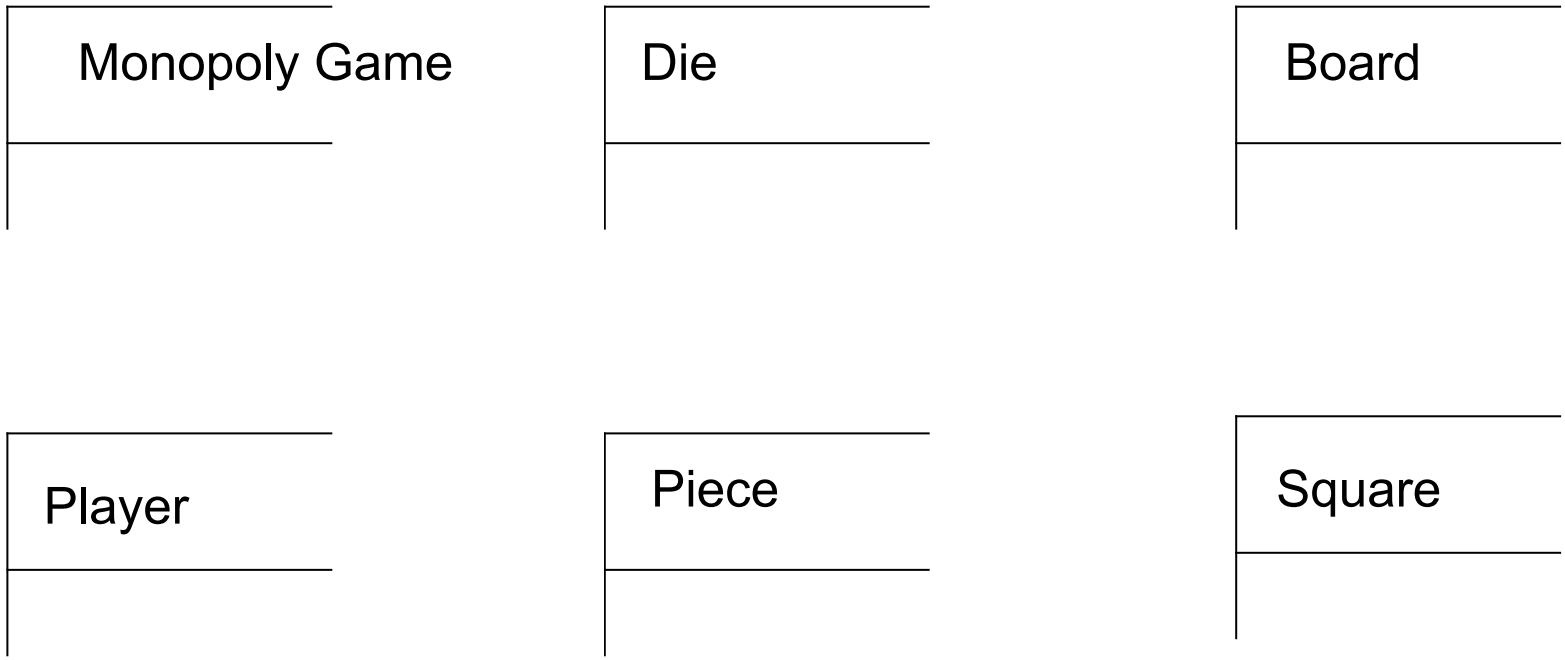
Marking Non-classes

- Monopoly game
- Player
- Board
- Square
- Die
- **Dice:** plural for Die
- Piece
- **Name**: attribute of Player / Piece / Square
- **Face value:** attribute for Die

Final Conceptual Classes

- Monopoly game
- Player
- Board
- Square
- Die
- Piece

Draw as UML classes



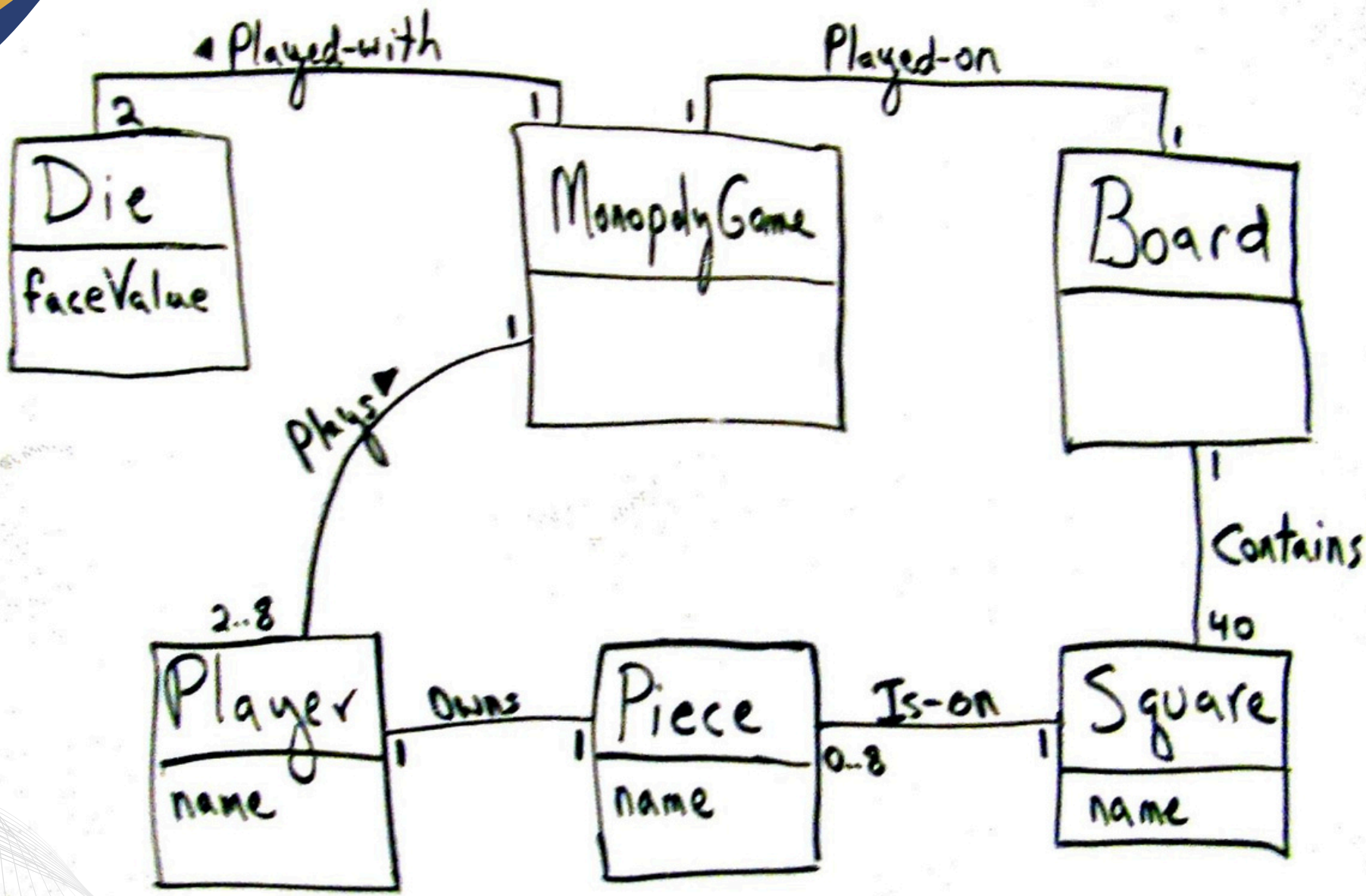
Step 4 & 5

Identify Associations

- A game is played with dice
- A game is played on a board
- A board contains squares
- Players play a game
- A player owns a piece
- A piece is on a square

Add Association Roles & Multiplicities

Monopoly Game domain model
Larman, Figure 9.28



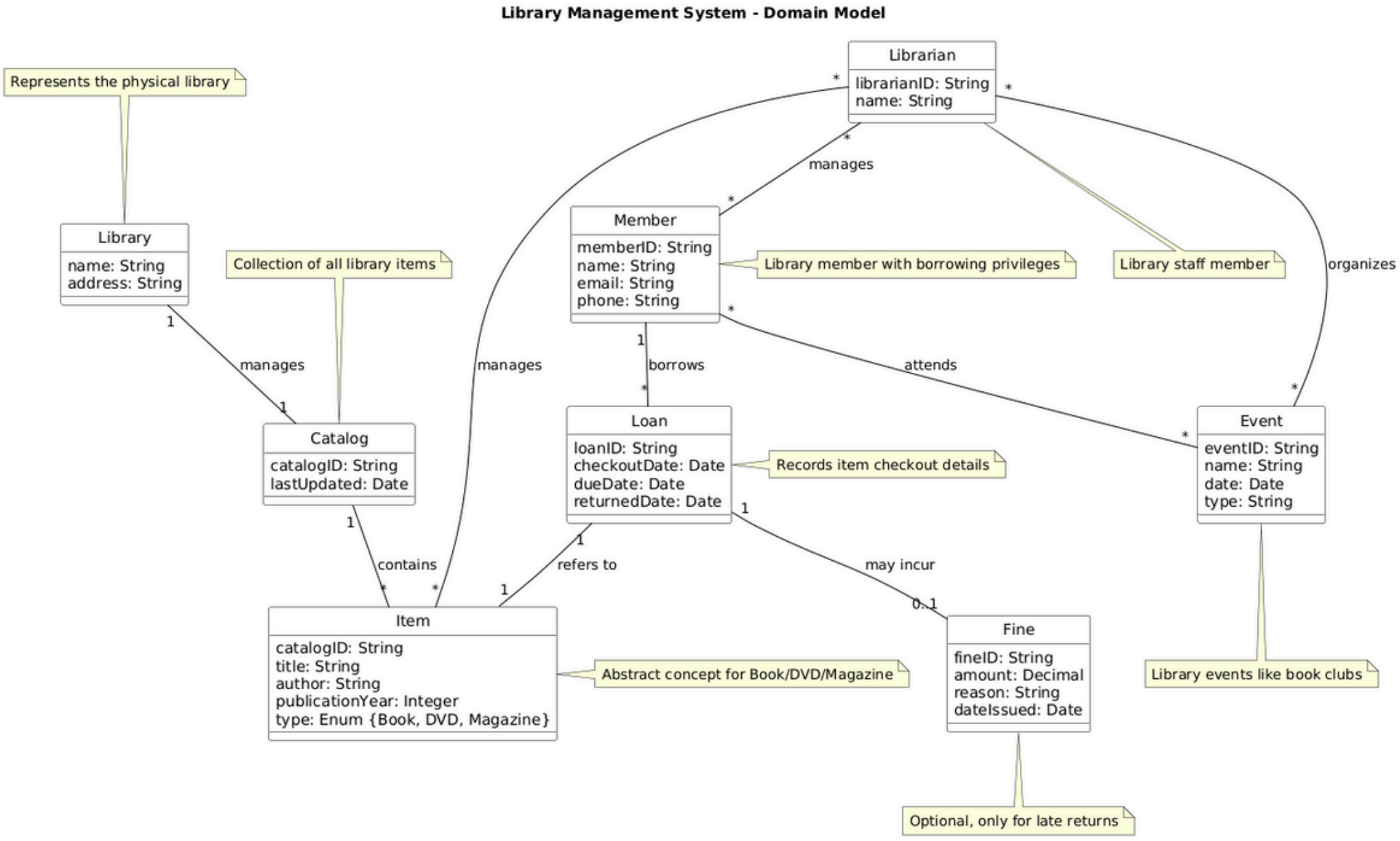
Exercise

A library allows members to borrow books, DVDs, and magazines. Each member has a unique ID, name, and contact information. The library catalog contains items, each with a title, author, publication year, and a unique catalog ID. Members can check out up to five items at a time for a loan period of two weeks. When an item is borrowed, the system records the checkout date and due date. Late returns incur fines, calculated per day per item. Librarians manage inventory, add new items, and handle member registrations. The library also hosts events, such as book clubs, which members can register to attend.

Exercise

- Identify candidate Conceptual Class
- Eliminate Non-class
- Add Attributes
- Identify Associations, also Add Association roles and multiplicities

Exercise: Solution



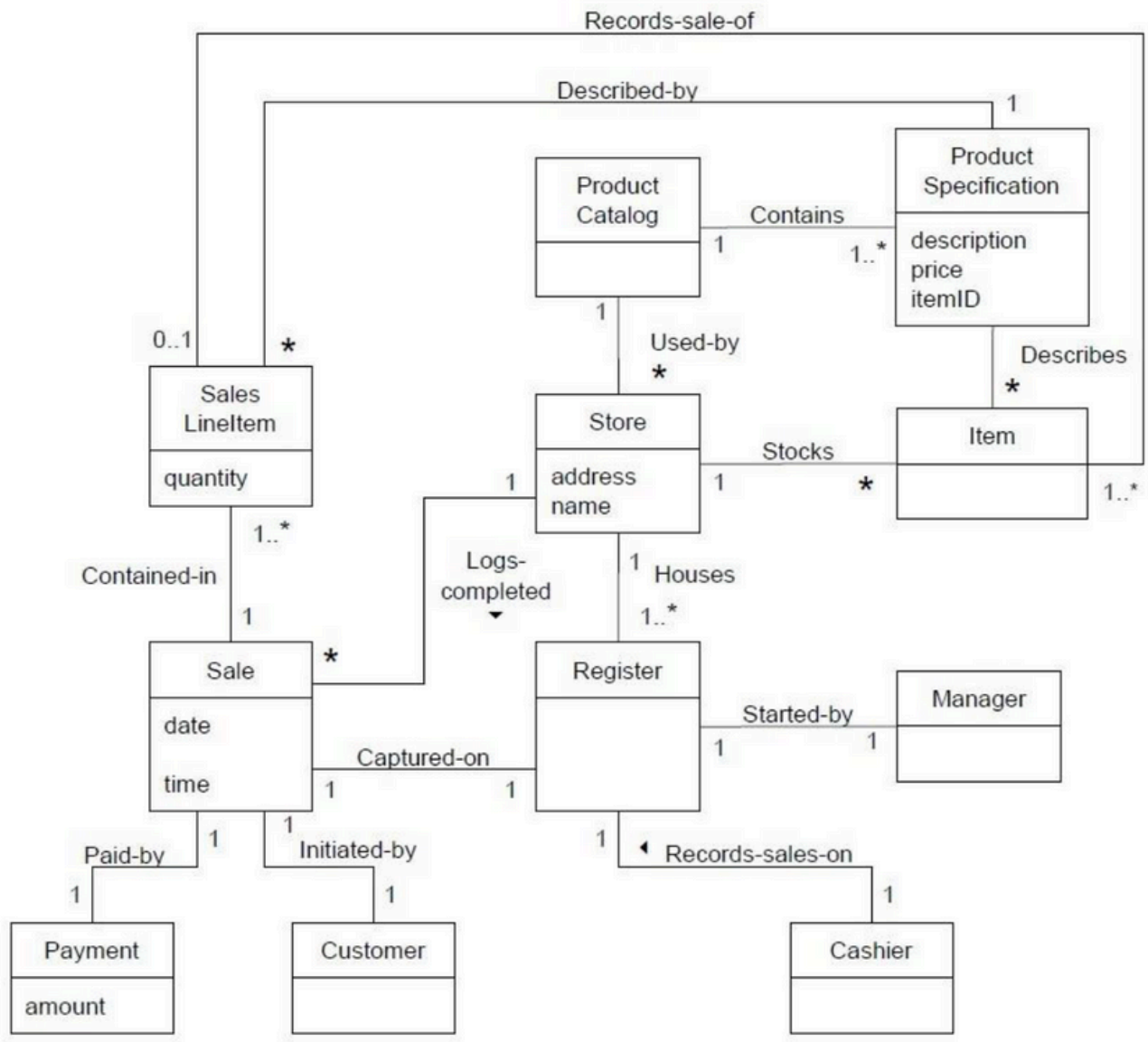
Point of Sale System (POS) [1]

A retail store maintains a product catalog containing specifications for all items sold, each with a description, price, and unique ID. The physical store at a specific location houses multiple point-of-sale registers. When a customer makes a purchase, a cashier initiates a new sale on a register, creating a sale record with date and time information. For each item purchased, the system creates a sales line item noting the quantity sold, which is linked to the corresponding product specification from the catalog.

Point of Sale System (POS) [2]

The store also tracks physical inventory items that are stocked on shelves. Once all items are recorded, the customer makes a payment for the total amount, completing the transaction. The system logs the completed sale, and managers can access sales records through the registers they oversee. This domain model captures the essential entities and relationships needed to manage sales transactions, inventory, and product information in a retail environment.

POS: Domain Model



THANK YOU